F – Agressive Cows

A distância máxima pode ser obtida verificando se para uma determinada distância é possível colocar todas as vacas nos "stalls", quando maior a distância mais difícil fica cumprir essa tarefa e quanto menor a distância mais fácil fica cumprir a tarefa, isso pode ser resolvido com busca binária fazendo a busca da distância no intervalo de 1 até o "stall" mais distante.

Complexidade: O(N(logN+log(MA))), onde MA é a distância do stall mais distante.

A – Cards

A cada nível são adicionadas 2*n (cartas dos triângulos) + n-1 (cartas de base pros triângulos de cima) cartas, como o limite é baixo é possível simular tudo em um vetor, mas também é possível adotar alguma estratégia com exponenciação binária (de números ou de matrizes)

Complexidade: O(N)

J – Menu

O problema basicamente quer o número de inversões de tamanho 3 no vetor, para isso só é preciso saber para cada número o número de elementos maiores que ele a esquerda e o número de elementos menores que ele a direita, o produto desses dois números é equivalente ao número de inversões com esse número no meio. Para achar o número de elementos maiores à esquerda ou menores à direita, pode-se usar uma BIT.

Link de apoio: https://www.geeksforgeeks.org/count-inversions-of-size-three-in-a-give-array/

Complexidade: O(NlogN)

K – Mixtures

Observe que a mistura final para cada intervalo contíguo independe das operações realizadas entre as misturas contidas no intervalo (apenas a fumaça muda), pode-se então precalcular uma matriz g de duas dimensões que segue a seguinte fórmula:

$$g(i,j) = (acum[j] - acum[i-1])\ mod\ 100$$

Logo o resultado do problema é F(0, N-1) onde:

$$F(i,j) = \begin{cases} 0, se\ i = j \\ \min\big(F(i,k) + F(k+1,j) + g(i,k) * g(k+1,j)\big), k\ indo\ de\ i\ até\ j-1 \end{cases}$$

Complexidade: $O(N^3)$

M - Serial and Parallel Resistors

Pode ser resolvido utilizando uma pilha para fazer o Expression Parsing da expressão dada.

Link de apoio: https://cp-algorithms.com/string/expression_parsing.html

Complexidade: O(N)

H – Remendo

Existem soluções puramente quadráticas para esse problema, a que será discutida terá um fator logN. Primeiramente ordena-se o vetor de furos, após isso é concatenado um vetor que é igual ao vetor de furos porém cada elemento é acrescido de C (para simular a volta no pneu), pra cada elemento do vetor original calcula-se o menor comprimento para ir de vet[i] até vet[i]+C, a resposta é o mínimo dentre todos elementos.

Complexidade: $O(N^2 logN)$

G – Mercado do Cairo

Basicamente, a ideia é começar dos mais velhos e verificar no retângulo que está ao mesmo tempo acima dele e à direita qual o melhor próximo vendedor que maximisse o número de itens que eu posso comprar, a resposta para dado vendedor é a resposta para o melhor próximo vendedor mais 1, para fazer essas consultas nos retângulos pode ser usada uma seg2D (também pode ser usada uma BIT2D pois ele vai sempre calcular até a ponta superior direita do grid, logo não é necessária a comutatividade).

Complexidade: $Nlog(max(x_i))log(max(y_i))$

L – Soldier and Traveling

Esse é um problema de fluxo máximo, cria-se arestas de capacidade igual ao tamanho original entre o source e os vértices, arestas de capacidade infinita entre os vértices e as cópias dos vértices que possuem aresta com esse vértice (também será necessário uma aresta ligando o vértice a sua cópia) e arestas de capacidade igual ao tamanho final entre as cópias dos vértices e o sink. Se o fluxo máximo for igual a soma dos valores originais e igual a soma dos tamanhos finais a resposta é YES e o valor pode ser recuperado sabendo por quais arestas houve fluxo. Caso contrário a resposta é NO.

Complexidade: Depende do algoritmo de fluxo utilizado

E – Caminhos Simétricos Ótimos

Primeiramente para retirar a restrição da simetria, basta somar o valor de cada elemento da matriz triangular superior ao seu respectivo simétrico da matriz triangular inferior. Feito isso basta utilizar um Dijkstra e descobrir o número de caminhos que vão do vértice inicial até a diagonal secundária da matriz, calculando também o número de caminhos mínimos, isso pode ser feito utilizando uma matriz auxiliar na etapa de relaxamento da aresta: se a distância atual pra o vértice v for igual a distância do vértice v mais o peso da aresta, o valor de v na matriz auxiliar recebe o valor de u. Se a distância de u mais o peso da aresta for igual a distância atual do vértice v, o valor de v na matriz auxiliar é acrescido do valor de u. Feito isso, verifica-se qual dos elementos da diagonal secundária tem o menor caminho e soma o número de possibilidades de todos os elementos da diagonal secundária que possuem o menor caminho.

Complexidade: $O(N^2 \log(N^2))$

D – Lasers

Basicamente a resposta é área do envoltório convexo que contém as projeções dos dos pontos na Terra pela direção do vetor do laser relativo a cada ponto.

Complexidade: O(NlogN)

C – Escaping from Escaping

A primeira coisa a se notar aqui é que o número de substrings de tamanho K de uma string é igual a N-K+1 onde N é o tamanho da string. Com o número de substrings possíveis para um tamanho K é 2^K, é fácil de perceber que a resposta no máximo vai ser 17, pois não é possível gerar todas as 2^{17} (131072) substrings. Como o TL é 7s, muitas estratégias são capazes de passar, uma bem fácil de se notar é gerar todas as substrings para um tamanho K e verificar se faltou aparecer alguma substring, se faltar a resposta é K, se não faltar deve-se testar com o tamanho K+1.

Uma solução bem legal (e uma das mais rápidas) é basicamente fazer uma BFS em uma árvore que contenha todos as substrings, o primeiro vértice que não possuir a sua lista de adjacência completa representa o nível da resposta (considerando o nível da raíz dessa árvore igual a 1). Não é possível gerar tal árvore, porém é possível gerar um DAG (Autômato de Sufixos) que é algo bastante parecido se for inserida a informação do nível que está sendo percorrido no DAG (como um grafo de estados) e aí é só realizar a BFS e interromper quando achar a resposta (muito importante porque se não dá TLE).

Complexidade: O(NlogN)

B – Fibonacci Polynomial

Não era a mais difícil, mas era necessário saber a relação:

$$f(n) = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n + \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$$

Multiplicando por x^n tem-se:

$$x^n f(n) = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}x\right)^n + \left(\frac{1-\sqrt{5}}{2}x\right)^n\right]$$

A soma desses termos pode ser vista como a soma de PGs infinitas (visto que como A existe, essas PGs convergem)

$$A = \sum_{n=0}^{\infty} x^n f(n) = \frac{1}{\sqrt{5}}\left(\frac{1}{1-\frac{1+\sqrt{5}}{2}x} - \frac{1}{1-\frac{1-\sqrt{5}}{2}x}\right)$$

$$A = \frac{1}{\sqrt{5}}\,\frac{\sqrt{5}x}{-x^2 - x + 1}$$

$$Ax^2 + Ax - A - x = 0$$

Resolve-se essa equação do segundo grau, para x ser racional o valor do discriminante tem ser que ser um quadrado perfeito, logo:

$$(A+1)^2 + 4A^2 \; tem\; que\; ser\; um\; quadrado\; perfeito$$

Complexidade: Fica a critério de vocês se achar a raíz de um número é O(1) ou O(logN). xD